PIANOGAN: A GENERATOR-CRITIC APPROACH FOR SYMBOLIC-DOMAIN MUSIC GENERATION

Edoardo Furlan†, Andrea Marigo†, Francesca Zorzi†

***Abstract*—In the field of generative AI, the task of music generation is heavily conditioned by the need of training a network that not only provides notes, but is able to produce pieces of music made of a coherent sequence of notes, and may be good for the human ear. Inspired by Midinet [1], our system is a conditioned Generative Adversarial Network (GAN) enhanced with a set of residual blocks and a dedicated Conditioner network that extracts features from previous bars to ensure temporal continuity. To produce a stable learning process, we adopt a Wasserstein loss with gradient penalty (WGAN-GP), replacing the standard binary cross-entropy. Also, the Critic (as we will call the Discriminator network, due to the new loss function) is enhanced with a Minibatch Standard Deviation layer, which provides information that helps in penalizing repetitive outputs. The integration of these systems is used to produce piano music from an underlying chord progression, generating coherent music.**

***Index Terms*—Neural Networks, Convolutional Neural Networks, Generative Adversarial Networks, Symbolic-Domain Music, Wasserstein distance, Residual blocks**

I. Introduction

The task of generating music is a frontier in Artificial Intelligence, since it poses the challenge of training a network to be creative, but with the rigid constraints imposed by music theory. While some architectures are already capable of generating technically correct sequences, capturing the key points of realistic, human-like compositions remains an interesting challenge. This is a particularly interesting problem if seen in the scenario of creating assistive tools for composers: both experienced and inexperienced musicians would benefit from these tools in order to improve creativity and exploration of genres. What we propose is an architecture that aims not only to model the placement of notes, but also the evolution of the themes they represent. In music generation, coherence is the main objective. For this reason, a simple RNN architecture seems to be the most appropriate solution, as its structure aims at learning sequences in order to predict the next values. However, MidiNet [1] showed that convolutional GANs can also be effective in this task, as the generation process is more creative due to the random noises given as input to the generator. However, in this kind of network, the main problem is that the discriminator tends to outperform the generator, rapidly reaching a high and stable accuracy. The network that we propose tries to enhance the capabilities of MidiNet by adding to the generation process a set of residual units that should help in learning long-term relations. Additionally, as mentioned in the MidiNet paper, to avoid the vanishing gradient problem, we changed the loss function from a conventional cross-entropy to a Wasserstein loss, specifically choosing the version that uses gradient penalty [2], [3]. In this scenario, the Discriminator works as a Critic, assigning a score of realism (not a probability) to the generated music. Finally, we also modified the critic network with an improved set of convolutional layers, to enhance its capability of extracting the relevant features from the music. These changes aim to improve the quality of the game played by the Generator and Critic, forcing more balanced capabilities in both structures, which should eventually allow for longer training and a more realistic result.

Our contributions can be summarized as follows:

- WGAN-GP (Wasserstein GAN - Gradient Penalty): loss is modified to help longer and more stable training.
- Residual Units: The generator is enriched with residual units to provide more direct flow of the gradient.
- Critic structure: the structure of the critic is improved to enable more effective feature extraction.
- Minibatch Standard Deviation: the critic is enriched with a minibatch standard deviation layer which provides statistical information about the produced music.

In the paper we follow this structure. In Section II we describe the base model we started from, in section III we explain the general processing pipeline. In Section IV we describe the dataset we used and the preprocessing techniques we adopted. In Section V we explain our model and we proceed in Section VI showing the results we obtained. Finally, some concluding remarks are provided in Section VII.

II. Related Work

Our network is based on the MidiNet architecture [1]. What this architecture introduced is the idea of conditioning the generation process by combining the outputs of convolutional layer of a Conditioner network to the input of the layers of the Generator. In this way the network is able to produce music coherent with previous bars. However, while training an exact copy of the network we noted the typical struggles of GANs: the Discriminator was quickly able to distinguish real from fake music, leading to a completely random Generation process. Our work then focused on finding ways to improve the balance between the two networks. The main idea was

†Department of Information Engineering, University of Padova, email: {edoardo.furlan.3, andrea.marigo.3, francesca.zorzi.3}@studenti.unipd.it

the modification of the loss function. As already said, the MidiNet paper opens to the possibiliy of changing to a loss that avoids vanishing gradients, which is the one we chose in our network.

III. Processing Pipeline

This section provides a high-level overview of the proposed processing pipeline, detailing the core functional blocks and their interrelations. Our system addresses the task of symbolic music generation by employing a conditioned Generative Adversarial Network (GAN) architecture. As illustrated in the system diagram (Figure 1), the overall pipeline revolves around three major interconnected components: a Generator, a Conditioner, and a Critic.

The primary objective of the network is to synthesize coherent and harmonically constrained musical passages represented as piano rolls. This is formulated as an adversarial minimax game: the Generator attempts to produce increasingly realistic music to deceive the Critic, while the Critic is trained to evaluate the realism of a piano roll.

The proposed framework is structured around the following underlying modules:

- **Generator Res-CNN:** The generative process is driven by a deep Residual Convolutional Neural Network. The model takes as input a random noise vector $z \in \mathbb{R}^{100}$ sampled from a standard Gaussian distribution, which provides the necessary stochasticity for generating diverse musical pieces. The Generator progressively upsamples this latent representation to yield the final two-dimensional structure encoding the musical bar.
- **Conditioner Network:** To ensure that the generated music adheres to a desired harmonic context, we adopt a conditioning mechanism inspired by [1]. The Conditioner is a CNN that processes a conditioning matrix representing the previous bars of the generated piece. Its intermediate feature maps are extracted and concatenated with the corresponding layers of the Generator. In doing so, the Conditioner influences and guides the generative process across multiple feature levels.
- **Critic CNN:** The Critic is designed as a Convolutional Neural Network. To counteract the well-known challenge of mode collapse in GANs, preliminary efforts involved adopting feature matching and one-sided label smoothing as suggested by [1]. Finding these measures insufficient for our specific task, we augmented our pipeline with a minibatch discrimination block. Specifically, this layer takes as input the feature map $I \in \mathbb{R}^{B \times C \times H \times W}$, where B is the batch size, C the number of channels, and H, W the spatial dimensions and computes the standard deviation across the batch dimension for each feature. These standard deviations are then averaged to produce a single scalar s, which represents the overall diversity of the current batch. This scalar is then concatenated to the original features. This strategy penalizes the Generator if it attempts to collapse onto a single repetitive mode, which is obviously seen as a symptom of mode collapse.

Detailed descriptions of each module, including layer configurations and the formalization of the adversarial objective (Section V-B), are provided in the following sections.

IV. Signals & Features

To generate coherent symbolic-domain music, our system relies on a pipeline that processes MIDI files into a highly structured matrix representation, suitable for training a Deep Convolutional Generative Adversarial Network (DCGAN). At a high level, the pipeline consists of data ingestion, feature extraction (chords and monophonic melodies), formatting, and adversarial training. While the generative architecture and the learning mechanism will be detailed in Section V, this section focuses on the dataset selection and the preprocessing methodologies required to map the MIDI data into effective neural network inputs.

A. Dataset Selection and Input Formatting

The dataset chosen to train our model is the Maestro V3.0.0 [4]. This collection provides over 200 hours of virtuosic piano performances, already processed and formatted as MIDI files. The primary motivation behind this choice is consistency: the dataset exclusively features piano performances, and the tempo is constant (120 bpm). This temporal uniformity is a crucial property, as it allowed us to parse all MIDI files using a constant sampling frequency. Consequently, we obtained a homogeneous data distribution that reduces the variance unrelated to the actual musical composition, thereby helping the model to better capture and reproduce the underlying melodic distribution.

Although MIDI files are structured, they are inherently event-based and highly polyphonic, making them unsuitable for direct ingestion into standard convolutional layers. Following the guidelines of related works in symbolic music generation [1], we mapped the data into a 2-D matrix (piano roll) representation.

Assuming a standard 4/4 time signature, we defined a spatial resolution of 16 time steps per bar and a pitch range of 128 discrete MIDI notes. The entire dataset was sequentially segmented into independent windows of 8 consecutive bars. Within our framework, two bars are considered temporally consecutive only if they belong to the same 8-bar segment. To enable the Generator to learn the onset of musical phrases without prior conditioning, the first bar of every segment is intentionally treated as the subsequent bar of an empty (silence) matrix.

B. Harmonic Feature Extraction

Since the Maestro dataset does not provide explicit chord labels, we used a dedicated feature extraction step to capture the underlying chord played. First, we computed Chroma features for each bar, aggregating the pitch energy across the 12 pitch classes. Then, by comparing the resulting Chroma vectors against predefined templates through cross-correlation, we assigned an integer index (from 0 to 24) to each bar. This index efficiently encodes 12 major chords, 12 minor chords, and a silence/no-chord state.

C. Signal Pre-processing

Raw piano rolls naturally feature complex polyphony, variable note velocities, and sparse activations. Since our target generative architecture is designed primarily to model monophonic melodies conditioned on harmonic contexts, several pre-processing steps were systematically applied to refine the signal, remove secondary artifacts, and enforce continuity:

1) **Monophonic Conversion and Prolonging:** The original piano rolls feature complex polyphony and sparse note activations. To extract the main melody, we applied a highest-note priority algorithm: for every time step, only the note with the highest pitch and velocity was retained, explicitly silencing the rest. Furthermore, to prevent the network from generating fragmented, staccato-like outputs, we employed a forward-fill strategy to bridge brief silences. If a time step was empty, the activation of the previous note was prolonged. This artificial continuity enforces legato phrasing, making it easier for the convolutional kernels to learn stable temporal dependencies.
2) **Binarization and Pitch Clipping:** To focus the learning process on the most musically expressive register and reduce data sparsity, each extracted segment was binarized (setting all non-zero velocities to 1) and pitch-clipped, zeroing out any note outside the $[24, 96]$ range.

Overall, this extraction and filtering procedure yielded a total of 44,123 valid segments, each formatted as a 128×16 matrix per bar.

D. Dataset Splitting

To evaluate the Generative Adversarial Network (GAN) rigorously and prevent data leakage, we adopted a hybrid dataset splitting approach built upon this homogeneous corpus:

- **Test Set:** Prior to any preprocessing or segmentation, a subset of three complete MIDI files from the Maestro dataset was strictly isolated. These completely unseen tracks are reserved exclusively for the final Conditioned Autoregressive Generation evaluation. Crucially, they provide the priming bar and the ground-truth harmonic progression required to properly constrain the generation process.
- **Training and Validation Sets:** The remainder of the musical corpus was processed into fixed-length matrices and randomly partitioned into a Training Set (90%) and a Validation Set (10%). The training set is utilized for weight optimization over a varying number of epochs, depending on the specific experiment. Concurrently, the validation set serves to monitor training stability over time.

V. Learning Framework

A. Model Architecture

The random noise vector, provided as input to the Generator G, is first processed through two fully-connected layers of 1024 and 512 neurons, respectively, and then reshaped into a tensor of shape $1 \times 256 \times 1 \times 2$. This is followed by four convolutional layers. At each stage, a chord embedding vector $\boldsymbol{c} \in \mathbb{R}^{12}$ is concatenated with the feature maps. The first three layers use filters of shape 1×2 with a stride of 2, effectively doubling the temporal dimension at each step. To mitigate the vanishing gradient problem and ensure training stability, we integrated a Residual Block after each upsampling layer. Finally, a transposed convolution with a 128×1 kernel, followed by a sigmoid activation function, produces the final bar of shape 128×16.

The Conditioner follows the structure described in [1]. Its primary objective is to enforce temporal continuity by conditioning the Generator on the previously generated bar. Structurally, it mirrors the Generator. The output of each layer in the Conditioner is concatenated with the corresponding feature maps in the Generator across its upsampling path.

The Critic is composed of four convolutional layers with filters of shape 4×4 and a stride of 2. The idea is to progressively extract features from the input bar, reducing the spatial dimensions from 128×16 to a 1×1 representation with 256 channels. Additionally, the minibatch standard deviation is included after the final convolutional stage. The resulting tensor is flattened, concatenated with the chord embedding $\boldsymbol{c}$, and subsequently passed through a fully connected layer that outputs a single scalar value. Notably, the standard sigmoid activation at the output of the Critic is omitted, as it is incompatible with the chosen loss function.

B. Wasserstein Loss and Gradient Penalty

To address the vanishing gradient problem, we adopted the Wasserstein GAN with Gradient Penalty (WGAN-GP) [2]. The idea is to minimize the Wasserstein distance between the real distribution P_r and the generated distribution P_g. This distance is estimated via a 1-Lipschitz function D (the Critic):

$$W(P_r, P_g) = \sup_{\|D\|_L \leq 1} \mathbb{E}_{\boldsymbol{x} \sim P_r}[D(\boldsymbol{x})] - \mathbb{E}_{\tilde{\boldsymbol{x}} \sim P_g}[D(\tilde{\boldsymbol{x}})], \quad (1)$$

where $D(\boldsymbol{x})$ is the scalar output of the Critic, which can be interpreted as a score assigned to the input sample $\boldsymbol{x}$.

The loss functions for the Critic, L_D, and the Generator, L_G, are formulated as

$$L_D = \mathbb{E}_{\tilde{\boldsymbol{x}} \sim P_g}[D(\tilde{\boldsymbol{x}})] - \mathbb{E}_{\boldsymbol{x} \sim P_r}[D(\boldsymbol{x})] \quad (2)$$

and

$$L_G = -\mathbb{E}_{\tilde{\boldsymbol{x}} \sim P_g}[D(\tilde{\boldsymbol{x}})], \quad (3)$$

respectively. By minimizing L_D, the Critic learns to provide an increasingly accurate estimation of the Wasserstein distance. On the other hand, by minimizing L_G, the Generator learns to produce music that receives higher scores, effectively pushing P_g closer to P_r.

To enforce the 1-Lipschitz constraint on the Critic, a gradient penalty is introduced, acting as a regularization term on the gradient norm. The total loss of the Critic is then computed

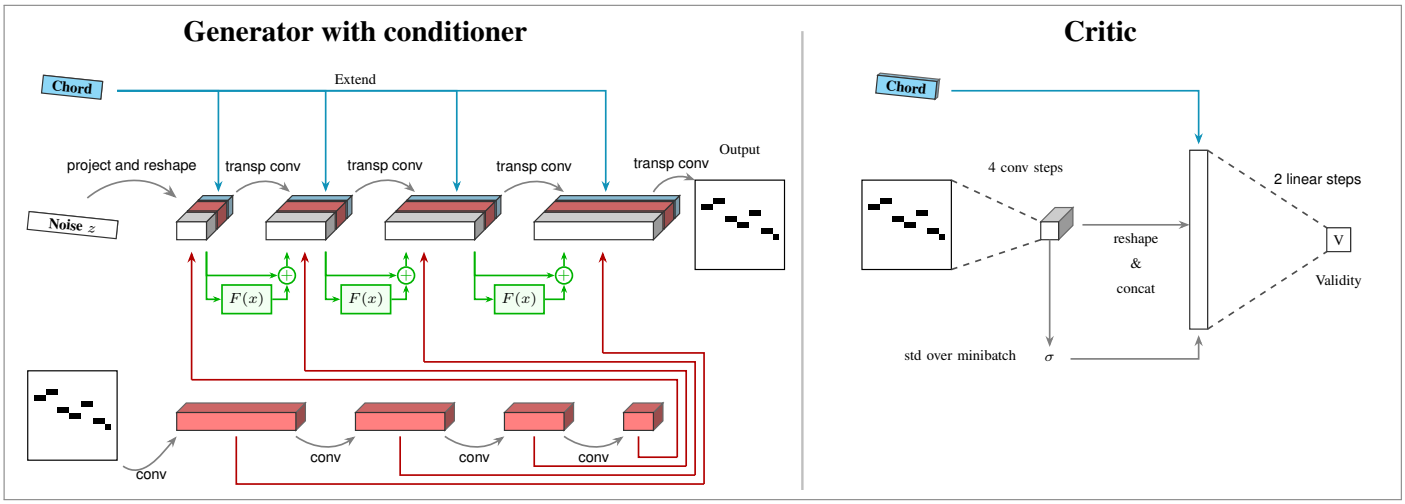


Fig. 1: Model architecture: Generator with conditioner (left) and Critic (right).

as

$$L_D^{\text{total}} = \underbrace{\mathbb{E}_{\tilde{\boldsymbol{x}}\sim P_g}[D(\tilde{\boldsymbol{x}})] - \mathbb{E}_{\boldsymbol{x}\sim P_r}[D(\boldsymbol{x})]}_{\text{Wasserstein loss}} + \underbrace{\lambda \mathbb{E}_{\hat{\boldsymbol{x}}\sim P_{\hat{\boldsymbol{x}}}}\left[(\|\nabla_{\hat{\boldsymbol{x}}} D(\hat{\boldsymbol{x}})\|_2 - 1)^2\right]}_{\text{Gradient penalty}}, \tag{4}$$

where $\hat{\boldsymbol{x}}$ is a sample drawn from a random interpolation between real and generated samples. It represents a random point along the straight line connecting $\boldsymbol{x}$ and $\tilde{\boldsymbol{x}}$. This regularization forces the Critic to maintain a gradient norm close to 1 along the trajectories connecting real and generated samples.

C. Hyperparameter Tuning

The training dynamics were primarily governed by three key hyperparameters:

1) **Gradient penalty coefficient (λ):** This parameter was kept constant at 10.0, following the recommendations in [2].
2) **Feature matching weight (w_{fm}):** The feature matching loss was employed as an additional objective to encourage the Generator to produce music matching the statistics of the real data, as perceived by the intermediate layers of the Critic. The total loss of the Generator is thus modified as follows:

$$L_G^{\text{total}} = L_{adv} + w_{fm} \cdot L_{fm}, \tag{5}$$

where L_{adv} is the adversarial loss,

$$L_{adv} = -\mathbb{E}_{\tilde{\boldsymbol{x}}\sim P_g}[D(\tilde{\boldsymbol{x}})], \tag{6}$$

and L_{fm} is the feature matching loss,

$$L_{fm} = \left\|\mathbb{E}_{\boldsymbol{x}\sim P_r}[f(\boldsymbol{x})] - \mathbb{E}_{\tilde{\boldsymbol{x}}\sim P_g}[f(\tilde{\boldsymbol{x}})]\right\|_2^2. \tag{7}$$

Here, $f(\cdot)$ represents the feature extractions from the Critic's convolutional layers, and w_{fm} is a weighting coefficient. Varying w_{fm} allows for controlling the trade-off between statistical accuracy and melodic variety.

3) **Number of Critic steps per Generator step (n_c):** This parameter was fundamental after the change of loss function and critic structure, as it allowed us to balance the training of the two networks. Empirical observations indicated that setting $n_c \in \{3, 4, 5\}$ was optimal. This provides the Critic with sufficient training iterations to deliver informative gradients to the Generator, thereby preventing mode collapse.

VI. Results

The results we show span the various architectures and methods we tried during the course of the project.

A. Baseline Model with BCE loss

In this first experiment we trained the baseline model with BCE loss. In Fig. 2 we show the curves of the accuracy of the Discriminator. We can see that after just a few epochs this curve directly went to 1, meaning that the Generator was never able to produce valid bars of music.

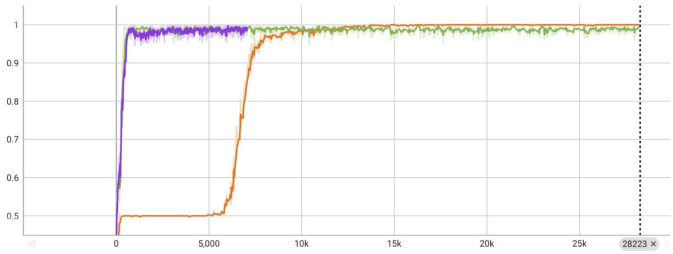


Fig. 2: Discriminator accuracy during the baseline training with BCE loss. The rapid convergence to 1 indicates that the Generator was immediately outperformed, leading to vanishing gradients.

The most delayed curve is the one corresponding to the introduction of the residual layers. Here the accuracy was able to stay around 0.5 for a few epochs, but then eventually collapsed to 1. At this point, the music produced was completely random.

B. Improved Model with WGAN-GP loss

The next step was the introduction of the WGAN-GP loss. Now, additionally to the generator and critic losses, we also tracked the *Wasserstein distance estimation* and the *feature matching loss*.

The idea of the first metric is that it should slowly decrease and stabilise towards zero, indicating indistinguishable distributions. Too high values indicate a too strong Critic, while too low negative values may indicate mode collapse, as the Critic is assigning too high values to the generated music

Regarding the feature matching loss, this also should slowly decrease towards low values, as it shows that the Generator is able to reproduce correctly the statistics of the real distribution. We first started with a weight of 5.0, but gradually decrised to 0.1 as we noticed that high values were favoring the model in reproducing average dataset statistics over diverse music, always leading to similar patterns (mode collapse). This means that in. the last trainings this metric was less used.

At this point, even if the game seemed more balanced, the Generator was always able to produce music that fooled the Discriminator, leading to an early mode collapse. It can be intersting to see how the loss of the generator always seemed to stay close to 0, with the critic loss being the one that fluctuated more. Looking at the produced music, we noticed that this was probably an indication of the Generator producing the same pattern over and over again, a clear signal of mode collapse. In Fig. 3 and Fig. 4 are shown the behaviour of the two losses during training.

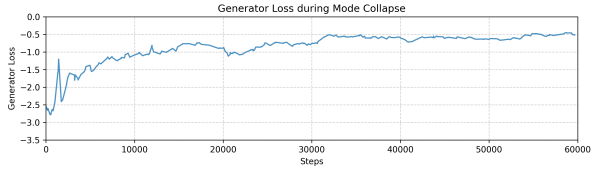


Fig. 3: Generator loss curve. It remains relatively stable and close to zero.

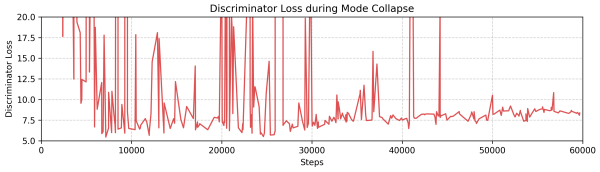


Fig. 4: Discriminator loss curve. The zoom on the [0, 20] range allows to observe the spikes and fluctuations with respect to the Generator loss.

Additionally, we also tried to produce some music at this point, as shown in Fig. 5. The main pattern is clearly a single set of close notes continuously repeated.

C. Improved Critic with minibatch std

At this point we tried to refine the Critic's architecture as the collapsing problem was probably caused by its inability

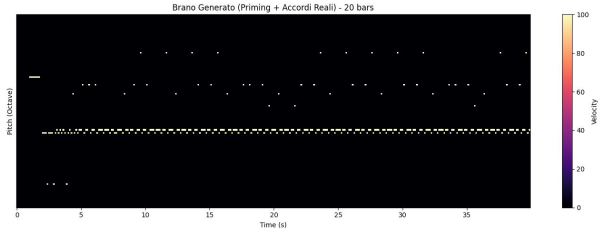


Fig. 5: Generated piano roll on one of the test music.

in extracting meaningful features from the produced music. We modified it into a deeper network, adding the stages we explained in V-A, with the objective of allowing it to extract more complex features from the music. Additionally, to force the Generator to produce more diverse music, we tried adding the minibatch standard deviation layer. This combination led to a more active learning curve regarding the Generator and Discriminator losses and an overall more stable behaviour of the Wasserstein distance. Here we also introduced the possibility for the Critic to perform a predefined number of updates for each update of the Generator, which allowed us to control more the balance between the two networks. In this final training it was se to 5.

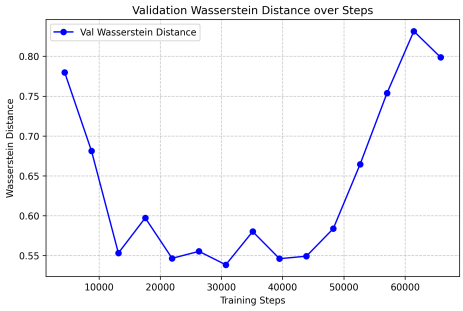
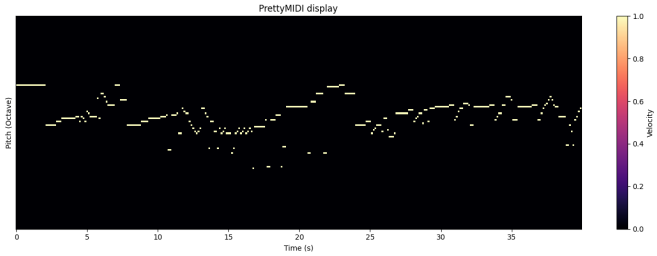


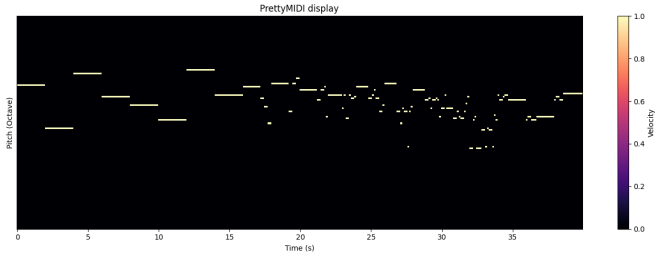
Fig. 6: Validation Wasserstein distance during training. The initial decrease shows the model learning the distribution, while the subsequent increase suggests a potential imbalance between the Generator and the Critic.

Shown in Fig. 6, is actually the behaviour of this metric on the validation set. It reveals an interesting training dynamic. This training is done for 15 epochs, with the feature matching weight set to 0, as we noticed it was not providing anymore useful results. Initially, the distance decreases, indicating that the Generator is successfully learning to approximate the real data distribution. However, after approximately 40,000 steps, this curve tends to increase, suggesting that the Critic is becoming significantly more effective than the Generator. This result shows that the model is capable of learning the real dynamics, but probably some fine tuning on the hyperparameters is still needed to reach a stable qeuilibrium and allow for longer trainings. The results that we show in the following figures are obtained with the model at a low point on the validation curve of the validation Wasserstein distance.

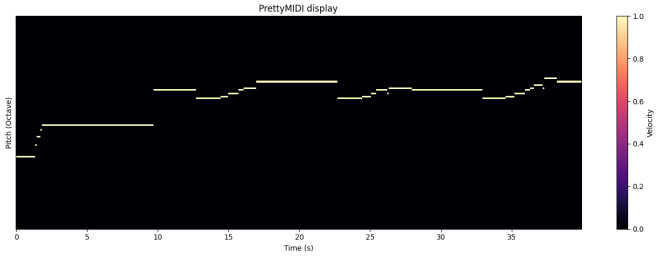
To obtain this result, we used one of the test samples from MAESTRO dataset and progressively produced 20 bars



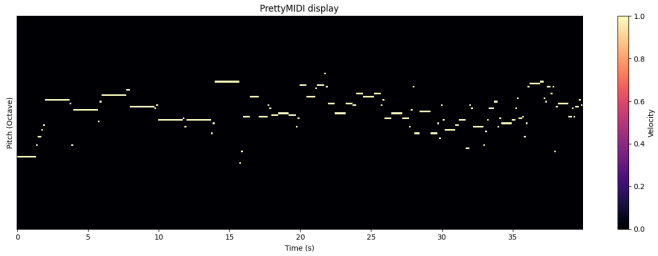
(a) Ground truth sample 1 from the MAESTRO dataset.



(b) Sample 1 generated by our final PianoGAN model.



(c) Ground truth sample 2 from the MAESTRO dataset.



(d) Sample 2 generated by our final PianoGAN model.

Fig. 7: Visual comparison between real and generated piano rolls. The model successfully captures the rhythmic density and the chordal structure defined by the input conditioning.

of music, each conditioned on the previously generated bar and on the underlying chord provided by the specific piece of music. The result is a music that seems to really be conditioned on the underlying chord progression and previous bars, with no mode collapse or random note generation, as can be seen in Fig. 7.

VII. Concluding Remarks

In this project we tried to tackle the problem of music generation by expanding the ideas already presented in [1]. We introduced a new way of setting the adversarial game, by using Wasserstein distance and gradient penalty instead of BCE loss. Furthermore, we gave a more profound structure to both the networks, providing them with a better understanding of the music generation process. We think that this network could be trained on different datasets and still be able to produce meaningful results, due to the various architectural constraints we imposed to the generation process. However, we also think that longer trainings would be needed to fully understand how the network behaves and how different parameter tunings could lead to better results.

What we realised by designing this network is that it is difficult to understand how the different blocks interact with each other and hence how to improve the model: each block we added or removed eventually led to a very different behaviour of the network, which was not always easy to predict. However, we understood that closely analyzing the losses and the results is the most important factor in understanding which are the required changes that should be made to the model. Each block we added had its own purpose and we could only evaluate its effectiveness by looking at these data.

References

[1] L.-C. Yang, S.-Y. Chou, and Y.-H. Yang, "MidiNet: A Convolutional Generative Adversarial Network for Symbolic-Domain Music Generation," in *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*, (Suzhou, China), Oct. 2017.

[2] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of wasserstein GANs," in *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS)*, (Long Beach, CA, USA), Dec. 2017.

[3] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, (Sydney, Australia), Aug. 2017.

[4] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the MAESTRO dataset," in *International Conference on Learning Representations*, 2019.